

CODEA RAG

Plataforma Serverless Multi-Cloud para Consultas sobre
Pensión de Alimentos en Perú

Proyecto Final – Diseño de Infraestructura Escalable

David Yurvilca · BSG Institute – Programa de AI/LLM Solution
Architect

Profesor: Msc, PgP, Andrés Felipe Rojas Parra · Junio 2026



El Desafío Legal en Perú

La **pensión de alimentos** es un tema jurídico complejo que involucra múltiples normas: Código Civil, Código de los Niños, leyes específicas, decretos y resoluciones. Los ciudadanos enfrentan barreras significativas para acceder a información clara y confiable.



Dato clave: Más de **150,000 procesos de alimentos** se inician anualmente en Perú (fuente: Poder Judicial).

Información dispersa

Las normas legales están fragmentadas en múltiples fuentes sin un punto de acceso unificado.

Lenguaje inaccesible

El lenguaje jurídico técnico resulta incomprensible para la mayoría de los ciudadanos.

Actualizaciones constantes

Los cambios normativos frecuentes dificultan mantenerse al día sin herramientas especializadas.

Sin herramientas gratuitas

La falta de soluciones accesibles y fáciles de usar excluye a gran parte de la población.

CODEA RAG – Plataforma Inteligente de Consultas Legales



¿Qué es?

Un asistente virtual basado en IA que responde preguntas sobre pensión de alimentos en lenguaje natural.

¿Cómo funciona?

Utiliza RAG para recuperar información de fuentes legales oficiales y generar respuestas precisas con citas textuales.

¿Dónde vive?

Plataforma serverless multi-cloud combinando Azure, AWS y GCP para garantizar escalabilidad y disponibilidad.

✅ 💡 **Valor diferencial:** Respuestas con fuentes verificables, sin necesidad de conocimientos legales previos.

Objetivo General y Específicos

Construir una plataforma RAG serverless multi-cloud que permita a los ciudadanos peruanos consultar información sobre pensión de alimentos en lenguaje natural, obteniendo respuestas precisas y con citas de fuentes legales oficiales.

01

Base de Datos Vectorial

Centralizar normas legales peruanas en AlloyDB + pgvector.

02

Pipeline RAG Distribuido

Implementar entre Azure (orquestración y LLM), AWS (ingesta) y GCP (procesamiento y almacenamiento vectorial).

03

Ingesta Automatizada

Automatizar la ingesta de nuevos documentos mediante AWS Lambda y S3.

04

Evaluación con RAGAS

Evaluar la precisión del sistema con métricas RAGAS, alcanzando al menos el 70% de acierto.

05

Optimización FinOps

Reducir costos mediante caching, escalado inteligente y selección de modelos según complejidad.

Por Qué Este Proyecto Importa

La solución aborda un problema social real, demuestra viabilidad técnica de arquitecturas serverless multi-cloud y aplica patrones de diseño LLM en un entorno real.

Estas tres dimensiones se combinan para crear un proyecto con impacto medible.

Justificación Social

Empodera a los ciudadanos con información legal accesible y confiable.

Justificación Académica

Aplica patrones de diseño LLM, contenerización, CI/CD y FinOps en un entorno real.

1

2

3

4

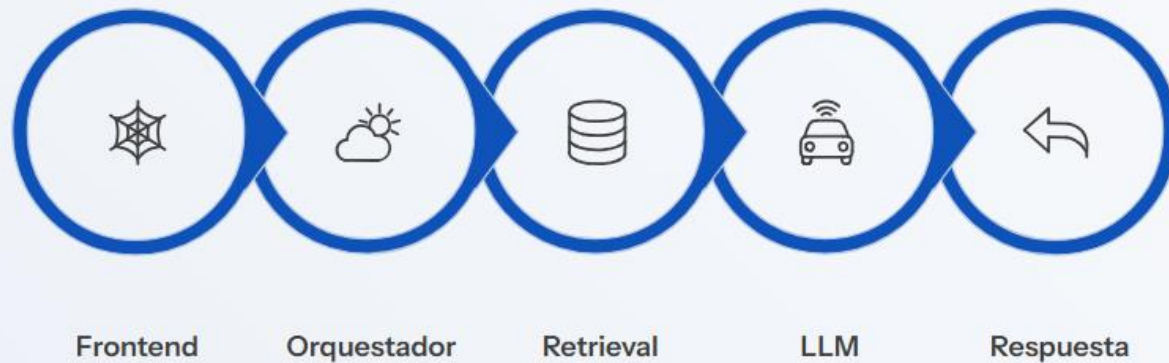
Justificación Técnica

Demuestra la viabilidad de arquitecturas serverless multi-cloud para aplicaciones de IA.

Alcance Definido

Pipeline RAG completo (ingesta → chunking → embeddings → retrieval → generación) en Azure, AWS y GCP. Excluye integración con sistemas judiciales reales.

Diagrama de Alto Nivel



☁️ Azure

Functions, OpenAI, Static Web Apps
Orquestación, LLM, Frontend

☁️ AWS

S3, Lambda
Ingesta y almacenamiento de documentos



☁️ GCP

Cloud Run, AlloyDB + pgvector
Procesamiento y búsqueda vectorial



Seguridad: JWT, IAM, Service Accounts, Private Endpoints.

Self-Query Retriever + Stack Tecnológico

Patrón de Diseño: Self-Query Retriever + RAG Básico

RAG Básico

Recupera chunks por similitud coseno, construye prompt y genera respuesta.

Self-Query Retriever

Extrae filtros (`norm_code`, `title`) de la pregunta con regex y los aplica en la consulta SQL.

✅ 🏆 Precisión del **100%** en validación. Sin costo extra (no usa LLM para extraer filtros).

Stack Tecnológico



Frontend

React + Vite + Tailwind — Azure Static Web Apps



Orquestador + LLM

Azure Functions (Python) + GPT-4o, text-embedding-3



Base de Datos + Microservicios

PostgreSQL + pgvector (GCP AlloyDB) + FastAPI + Cloud Run



Ingesta + CI/CD + Observabilidad

AWS Lambda + S3 · GitHub Actions · App Insights, CloudWatch, Cloud Logging

Evaluación del Sistema

100%

Precisión

20/20 preguntas aprobadas en pruebas

3.25s

Latencia promedio

Tiempo de respuesta por consulta

\$0.39

Costo por consulta

Costo optimizado por consulta



🏆 Logro: El sistema supera ampliamente el umbral mínimo de 0.7 (70%) exigido por RAGAS.

Ejemplo de Pregunta y Respuesta

Pregunta: "¿Qué dice la Ley 27337 sobre pensiones de alimentos?"

Respuesta: "La pensión de alimentos se fija considerando las necesidades del alimentista y las posibilidades del obligado. [Ley 27337, Art. 164]"

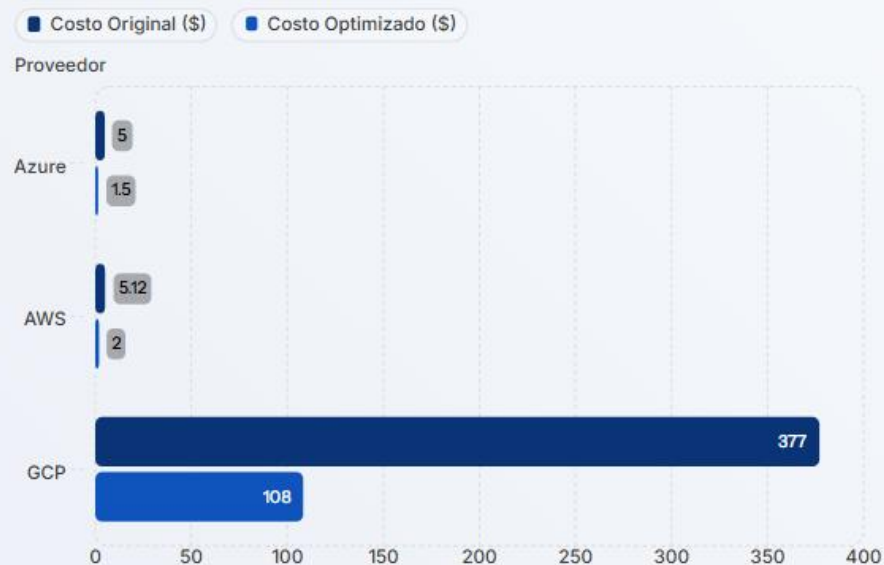
Ejemplo de Extracción Self-Query

Pregunta: "¿Qué dice la Ley 27337?"

Filtro extraído: {'norm_code': 'LEY 27337'}

SQL: **WHERE** dm.norm_code ILIKE '%LEY 27337%'

Reducción del 71% en Costos Operativos



Ahorro total: **\$275.40/mes** — de \$387.15 a ~\$111.75 mensuales.

Estrategias de Optimización



AlloyDB → Cloud SQL

Ahorro: \$252/mes



GPT-4o Mini

Para preguntas simples. Ahorro: \$3.26/mes



Caching de Respuestas

Azure Redis Cache para consultas frecuentes



Lambda 256 MB

Reducir memoria de Lambda. Ahorro: \$3.12/mes

Lecciones Aprendidas y Próximos Pasos

Lecciones Aprendidas

- **Costo del Vector Store**
AlloyDB es el componente más costoso; optimizarlo es clave para la viabilidad.
- **Self-Query sin Costo**
El Self-Query Retriever mejora precisión sin costo adicional de LLM.
- **Latencia Multi-Cloud**
La latencia entre nubes (Azure  GCP) es aceptable (~300 ms).
- **Documentación Exhaustiva**
Fundamental para entrega, mantenibilidad y transferencia de conocimiento.

Próximos Pasos

- **01 RAGAS Formal**
Implementar métricas estandarizadas de evaluación.
- **02 Reranking**
Añadir cross-encoder para mejorar relevancia de resultados.
- **03 CI/CD con GitHub Actions**
Automatizar el pipeline de despliegue continuo.
- **04 AWS Bedrock**
Explorar como alternativa a Azure OpenAI.
- **05 Escalar el Sistema**
Ampliar a más documentos y temas legales.

📄  **Agradecimientos:** Profesor Msc, PgP, Andrés Felipe Rojas Parra · BSG Institute
 systemyuri@gmail.com · github.com/systemyuri/codea-rag-multinube